backroom

Cafe Simulator Inventory and Environment Module.

This module defines the classes for managing the backroom storage system, including shelving units, individual shelf spots, and ingredient containers.

• View Source

class StockingShelf(constants.GameObject): • View Source

A large shelving unit that manages multiple individual storage spots.

Attributes: interaction_zone (pygame.Rect): The area where a player must stand to interact. spots (list): A collection of shelf_spot objects contained within this shelf. icon (pygame.Surface): The visual sprite for the shelf unit.

StockingShelf(x, y, w, h) • View Source

Initializes the shelf and generates its internal grid of storage spots.

interaction_zone

spots

icon

def placeshelf_spot(self, num): • View Source

Updates given shelf spot

def render(self, screen, font, DebugMode): • View Source

Draws the shelf icon and triggers the render method for all nested spots.

Inherited Members

constants.GameObject color, rect

class shelf_spot(constants.GameObject): • View Source

An individual slot on a shelf that can hold a single ingredient box.

Attributes: open (bool): Whether the spot is currently empty. held_ingredient_box (ingredientBox): The box object currently stored in this spot.

shelf_spot(x, y, w, h, parent) • View Source

Initializes an empty shelf spot.

open

held_ingredient_box

parent

def store_ingredient_box(self, player): • View Source

Transfers an ingredient box from the player's active inventory slot to this spot.

Args: player (Player): The player object attempting to store an item.

def remove_ingredient_box(self): • View Source

Resets box to open state

def render(self, screen, font): • View Source

Renders its contained box if it is currently occupied.

Inherited Members

constants.GameObject color, rect

class IngredientBox(constants.GameObject): • View Source

A container for raw ingredients with a limited quantity.

Attributes: ingredient (Ingredient): The type of ingredient stored inside. quantity (int): Remaining units before the box is depleted. interaction_zone (pygame.Rect): Clickable area when the box is on the floor.

IngredientBox(x, y, ingredient) • View Source

Initializes the box with a specific ingredient and a default quantity of 10.

ingredient

quantity

interaction_zone

name

spot

icon

ingredient_icon

stackable

def update_position(self, center): • View Source

Updates the physical coordinates of the box to align with a shelf spot's center.

def pop_box(box, ingredient_boxes, backroomCollisions): • View Source

Static helper to remove a box from the global game tracking lists.

Args: box (ingredientBox): The box instance to remove. ingredient_boxes (list): The list of all boxes in the room. backroomCollisions (list): The list of active collision rects.

def pick_ingredient(ingredients_list, slot_index=**None**): • View Source

Returns a guaranteed ingredient per slot so essential items always spawn. Slot assignments: 0 -> Coffee Beans (index 0) 1 -> Water (index 3) 2 -> Ice (index 5) 3 -> Milk (index 6) 4 -> Cocoa Powder (index 9)

def set_spot(self, spot): • View Source

Links this box to a specific shelf spot.

def grab_ingredient(self, player): • View Source

Removes one unit from the box and adds it to the player's inventory.

Args: player (Player): The player retrieving the ingredient.

def place_ingredient_in_box(self, player): • View Source

Removes one unit from the players inventory and adds it a box.

Args: player (Player): The player storing the ingredient

def render(self, screen, font, DebugMode=**False**): • View Source

Draws the box icon, ingredient icon, and interaction zone if applicable.

Inherited Members

constants.GameObject color, rect

class DoorEntry(constants.GameObject): • View Source

A floor rug representing a transition point between different cafe rooms.

DoorEntry(x, y, w, h) • View Source

Initializes the entry point with a rug graphic.

icon

icon_rect

def render(self, screen): • View Source

Renders the entry rug at its designated coordinates.

Inherited Members

constants.GameObject color, rect

class Refrigerator(StockingShelf): • View Source

A Refrigerator unit that manages two storage spots for cold ingredients

Attributes: ...

Refrigerator(x, y, w, h) • View Source

Initializes the refrigerator object for back room storage

interaction_zone

spots

icon

Inherited Members

StockingShelf placeshelf_spot(), render()
constants.GameObject color, rect

button

• View Source

class Button: • View Source

A clickable UI element used for navigation or triggering game actions.

This class handles rendering text on a rectangle and detecting mouse interactions for state changes.

Button(x, y, width, height, text, target_state, action) • View Source

Initializes the button with its position, dimensions, and behavior.

Args: x (int): Horizontal position of the button. y (int): Vertical position of the button. width (int): Width of the button rectangle. height (int): Height of the button rectangle. text (str): The label displayed on the button. target_state (str): The game state to transition to when clicked. action (function): A callback function to execute on click.

rect

text

target_state

font

color

hover_color

text_color

action

button_sound

def is_clicked(self, mouse_pos): • View Source

Checks if a given mouse position is within the button's boundaries.

def draw(self, screen): • View Source

Renders the button to the screen, changing color if the mouse is hovering.

Args: screen (pygame.Surface): The display surface to draw on.

def handle_event(self, event, current_state): • View Source

Processes mouse events to determine if the button was clicked.

Returns the target_state if clicked, otherwise returns the current_state.

constants

• View Source

FPS = 60

PLAYER_SPEED = 6

CUSTOMER_SPEED = 2

CUSTOMER_SPAWN_EVERY_MS = 4000

MAX_CUSTOMERS = 10

MAX_CUSTOMERS_WAITING = 4

ORDER_DELAY = 4000

Colors

TABLE_COLOR = (140, 110, 70)

REGISTER_COLOR = (0, 0, 0)

COUNTER_COLOR = (140, 110, 70)

SEAT_COLOR = (0, 0, 0)

NPC_COLOR = (255, 220, 80)

UI_BG_COLOR = (30, 30, 30)

YELLOW = (255, 215, 0)

BLUE = (80, 160, 255)

GREEN = (0, 200, 0)

WHITE = (255, 255, 255)

BLACK = (0, 0, 0)

ORDER_COLORS = [(255, 215, 0), (80, 160, 255)]

IMAGE_LIBRARY = show

LINE_POSITIONS = [(840, 350), (660, 335), (490, 320), (320, 305)]

CUSTOMER_ENTRY_X = -130

CUSTOMER_ENTRY_Y = 315

REAL_DAY_SEC = 86400

TIME_SPEED = 72

DAY_START = 21600

SEVEN_AM = 25200

EIGHT_AM = 28800

FOUR_PM = 57600

DAY_END = 64800

EIGHT_PM = 72000

MAX_INGREDIENT_BOXES = 4

BOX_POSITIONS = [(625, 660), (750, 660), (875, 660), (1000, 660)]

NUM_SLOTS = 4

SLOT_SIZE = 50

INVENTORY_POSITIONS = [(10, 500), (10, 560), (10, 620), (10, 680)]

RECIPE_VIEW_NONE = None

RECIPE_VIEW_MENU = 'MENU'

RECIPE_VIEW_DETAIL = 'DETAIL'

RECIPE_ICON_SIZE = (100, 100)

RECIPE_ICON_PADDING = 20

RECIPE_START_X = 200

RECIPE_START_Y = 200

OPEN_RECIPE_MENU_KEY = 109

CLOSE_MENU_KEY = 27

Cafe Game Base Classes Module. Defines core game objects and ingredient logic for the cafe simulation.

class GameObject: • View Source

A base class for all interactable and rendered entities in the game.

Attributes: x (int): Horizontal coordinate. y (int): Vertical coordinate. w (int): Width of the object. h (int): Height of the object. rect (pygame.Rect): The collision and boundary rectangle. color (tuple): RGB color value for default rendering.

GameObject(x, y, w, h, color) • View Source

Initializes position, dimensions, and the pygame Rect object.

color

rect

def render(self, screen): • View Source

Draws a simple rectangle to the screen if no sprite is provided.

class Counter(GameObject): • View Source

Defines the visual and placeable part of a counter unit.

Attributes: placeable (bool): Whether objects can be placed on this unit.

Counter(x, y, w=150, h=90, color=(140, 110, 70)) • View Source

Initializes position, dimensions, and the pygame Rect object.

placeable

Inherited Members

GameObject color, rect, render()

class Window(GameObject): • View Source

A base class for all interactable and rendered entities in the game.

Attributes: x (int): Horizontal coordinate. y (int): Vertical coordinate. w (int): Width of the object. h (int): Height of the object. rect (pygame.Rect): The collision and boundary rectangle. color (tuple): RGB color value for default rendering.

Window(x, y, w, h) • View Source

Initializes position, dimensions, and the pygame Rect object.

icons

current_image

def update_current_image(self, day_type): • View Source

```
def render(self, screen, game_seconds):
```
• View Source

Draws a simple rectangle to the screen if no sprite is provided.

Inherited Members

GameObject color, rect

customer

• View Source

```
class Customer(constants.GameObject, pygame.sprite.Sprite):
```
• View Source

An NPC that orders recipes, waits in line, and sits at tables.

Attributes: sprite (pygame.Surface): The current visual image of the customer. recipes_unlocked (list): The list of available recipes the customer can pick from. ordered_item (Recipe): The specific recipe the customer has ordered. state (str): The current behavior state (waiting, finding seat, etc.). wait_bar_length (int): The current satisfaction/timer value.

```
Customer(x, y, image_keys, recipes_unlocked_list, line_position)
```
• View Source

Initializes a customer with a random order and a designated line position.

Args: x (int): Starting horizontal position. y (int): Starting vertical position. image_keys (list): Keys for retrieved sprites from IMAGE_LIBRARY. recipes_unlocked_list (list): List of Recipe objects available for ordering. line_position (tuple): The (x, y) coordinate for the customer's spot in line.

image_keys

image

rect

recipes_unlocked

ordered_item

state

target_seat

target_position

line_position

wait_bar_length

drink_start_time

drink_duration

serve_result

assigned_cup

def pick_item(self): • View Source

Picks a random item from the list of unlocked recipes.

Returns: Recipe: The randomly selected recipe object.

def find_seat(self): • View Source

Moves the customer toward a target seat and occupies it upon arrival.

def move_toward_point(self, target_x, target_y): • View Source

Move toward a point smoothly. Returns True when the point is reached.

Args: target_x (int): Horizontal target coordinate. target_y (int): Vertical target coordinate.

def move_up_in_line(self): • View Source

Move smoothly to the customer's next assigned line position.

def start_drinking(self, result): • View Source

Put the customer into the drinking state after being served.

Args: result (str): Either "correct" or "incorrect".

def leave_cafe(self): • View Source

Moves the customer toward the exit coordinate off-screen.

def set_state(self, state): • View Source

Sets the customer's behavioral state.

def set_target_seat(self, seat): • View Source

Assigns a target seat and sets a waypoint in front of it.

def render(self, screen, debugmode=**False**): • View Source

Renders customer and satisfaction bar.

def calculate_tip(self): • View Source

Calculate tip based on remaining wait bar value.

Returns: tuple: (base_pay, tip, total)

def update(self, collisions): • View Source

Updates the customer's behavior every frame using a state machine.

Args: collisions (list): List of collision boundaries (reserved for future use).

def get_foot_rect(self): • View Source

Returns a rectangle that only covers the feet area of the customer sprite.

Inherited Members

constants.GameObject color

furniture

• View Source

class Table(constants.GameObject): • View Source

A furniture object that holds Seat objects.

Attributes: seats (list): The list of individual Seat objects associated with this table. open (bool): Whether the table has available seats.

Table(x, y, w=70, h=40, color=(140, 110, 70), seats=**None**) • View Source

Initializes position, dimensions, and the pygame Rect object.

open

seats

Inherited Members

constants.GameObject color, rect, render()

class Seat(constants.GameObject): • View Source

A specific seating spot for a customer.

Attributes: state (str): Current occupancy status (open, reserved, taken). seated_customer (Customer): The customer assigned to this seat. num (int): The unique seat ID for orientation and identification.

Seat(x, y, num, w=70, h=15, color=(0, 0, 0)) • View Source

Initializes position, dimensions, and the pygame Rect object.

state

seated_customer

num

def get_seat_x(self): • View Source

Returns the horizontal position of the seat.

def get_seat_y(self): • View Source

Returns the vertical position of the seat.

def reserve_seat(self, customer): • View Source

Sets seat state to reserved for a specific customer.

```
def occupy_seat(self, customer):
```
• View Source

Sets seat state to taken by a specific customer.

```
def open_seat(self):
```
• View Source

Resets the seat to an open state.

Inherited Members

constants.GameObject color, rect, render()

game

• View Source

screen = <Surface(1366x768x32 SW)>

MUSIC_GROUPS = show

correct_order = <pygame.mixer.Sound object>

incorrect_order = <pygame.mixer.Sound object>

bag_coffee_beans = <items.Ingredient object>

ground_coffee = <items.Ingredient object>

espresso_shot = <items.Ingredient object>

espresso_doubleShot = <items.Ingredient object>

water = <items.Ingredient object>

hot_water = <items.Ingredient object>

ice = <items.Ingredient object>

milk = <items.Ingredient object>

steamed_milk = <items.Ingredient object>

foamed_milk = <items.Ingredient object>

cocoa_powder = <items.Ingredient object>

chocolate_mix = <items.Ingredient object>

INGREDIENTS = show

Espresso = <recipes.Recipe object>

Iced_Coffee = <recipes.Recipe object>

Americano = <recipes.Recipe object>

Latte = <recipes.Recipe object>

Iced_Latte = <recipes.Recipe object>

Hot_Chocolate = <recipes.Recipe object>

ALL_RECIPES = show

RECIPES_UNLOCKED = []

counter3_rect = <rect(0, 590, 983, 50)>

wall_rect2 = <rect(0, 293, 1400, 10)>

backroom_right_wall = <rect(1280, 0, 100, 800)>

counter1_rect = <rect(187, 336, 983, 50)>

counter2_rect = <rect(187, 718, 983, 50)>

wall_rect = <rect(0, 333, 1400, 10)>

menu_rect = <rect(1150, 0, 100, 800)>

back_wall_rect = <rect(0, 320, 1400, 10)>

windows = show

register1 = <others.Register object>

register2 = <others.Register object>

customer_was_waiting = False

currentCust = None

currCustomer = None

current_screen = 'game'

front_collisions = [<rect(1150, 0, 100, 800)>, <rect(0, 590, 983, 50)>, <rect(0, 293, 1400, 10)>]

middle_collisions = show

backroom_collisions = show

front_counters = show

seats = show

middle_counters = show

sink = <others.Sink object>

grinder = <Machine Sprite(in 0 groups)>

espresso_mach = <Machine Sprite(in 0 groups)>

water_boiler = <Machine Sprite(in 0 groups)>

cocoa_station = <Machine Sprite(in 0 groups)>

ALL_MACHINES = show

machines = []

SHOP_VIEW_NONE = None

SHOP_VIEW_MENU = 'MENU'

recipe_button = <button.Button object>

shop_button = <button.Button object>

menu_start_button = <button.Button object>

pause_resume_button = <button.Button object>

pause_quit_button = <button.Button object>

next_day_button = <button.Button object>

quit_button = <button.Button object>

camera_button_rect = <rect(750, 40, 40, 40)>

music_button_rect = <rect(820, 40, 40, 40)>

music_menu_rect = <rect(820, 85, 140, 120)>

music_option_rectangles = [<rect(830, 95, 120, 30)>, <rect(830, 130, 120, 30)>, <rect(830, 165, 120, 30)>]

delete_save_button1 = <button.Button object>

delete_save_button2 = <button.Button object>

delete_save_button3 = <button.Button object>

save1_button = <button.Button object>

save2_button = <button.Button object>

save3_button = <button.Button object>

save_game1 = {'file': 'save1.json', 'button': <button.Button object>}

save_game2 = {'file': 'save2.json', 'button': <button.Button object>}

save_game3 = {'file': 'save3.json', 'button': <button.Button object>}

current_save_file = None

all_save_files = show

switch_view_prompt_rect_cafe = <rect(1025, 675, 100, 100)>

switch_view_prompt_rect_middle = <rect(50, 675, 100, 100)>

class GameManager: • View Source

Holds game-wide progression and UI state.

This keeps economy, orders, messages, and placeholder upgrade/shop data together without changing the gameplay loop structure too much.

GameManager() • View Source

Initialize the shared game state.

message

message_timer

machine_loaded_slot

active_orders

max_orders

more_hands_tier

music_menu_open

current_music_group

current_song_index

money

money_earned_today

day_num

num_customers_today

customers_unhappy_today

name_input_box

name_prompt

name_input_text

name_input_active

shop_tabs

upgrades

active_tab

shop_page

ITEMS_PER_PAGE

tab_order

shop_tab_rects

shop_item_rects

shop_item_orig_idx

shop_arrow_left_rect

shop_arrow_right_rect

placement_machine

placement_item

recipe_active_tab

recipe_page

selected_recipe

recipe_cards_per_row

recipe_rows_per_page

recipe_tab_rects

recipe_card_rects

recipe_card_map

recipe_arrow_left_rect

recipe_arrow_right_rect

save_name

data

def set_message(self, text, duration_ms=1500): • View Source

Set a temporary on-screen message.

Parameters: text (str): The message to display. duration_ms (int): How long to show the message in milliseconds.

def update_message(self): • View Source

Clear the current message once its timer expires.

def clear_round_state(self): • View Source

Reset lightweight gameplay state used during testing.

def buy_upgrade(self, upgrade_index): • View Source

Attempt to buy an upgrade using the old list index.

Kept so it works with the older stuff — talks to buy_shop_item() using the Upgrades tab as the target. upgrade_index (int): The index of the upgrade in the Upgrades tab list.

def buy_shop_item(self, tab_name, item_index, player=**None**): • View Source

Attempt to purchase an item from the given shop tab.

Handles tier-gating for the Upgrades tab (higher tiers require the previous tier to be purchased first). All other tabs treat items as independently purchasable with no lock order.

Parameters: tab_name (str): The tab the item lives in (e.g. "Upgrades"). item_index (int): The absolute index of the item within that tab's list. player (Player): The player instance, required for upgrades that directly modify player attributes/stats. Optional for upgrades that only modify GameManager state or are purely cosmetic.

def handle_time(self, hrs, mins): • View Source

Takes in the game's hours and minutes and converts them to follow standard clock rules while on a 5 minutes interval.

def findFirstOpen(self, seats): • View Source

Finds first open seat in collisions list and returns it. None if all occupied.

def get_nearby_seated_customer(self, player, customers): • View Source

Return the first seated customer close enough to serve.

Uses the same general 'nearby interaction' style as machines.

def cleanup_gone_customers(

```
    self,
    customers,
    customersWaiting,
    all_sprites,
    customer_group,
    manager,
    player
):
```
• View Source

Remove customers that have fully left the cafe. Also remove their resolved order cards once they are gone.

```
def drawHotBar(self, player, font):
```
• View Source

Draw the player's inventory hotbar on the left side of the screen.

Uses the fixed INVENTORY_POSITIONS list from constants to position each slot.

```
def draw_recipe_screen(self, screen):
```
• View Source

Draw the recipe menu overlay with two tabs (Unlocked / Locked), a card grid of recipes, pagination, and an info bar.

Layout (top to bottom): - Floating "Recipes" title above the box (main-menu gold style) - Box: tabs → info bar → card grid → dot indicator - Pagination arrows centered vertically on the left/right box edges

Clicking an unlocked card opens the detail view (RECIPE_VIEW_DETAIL). Clicking a locked card deducts money and moves it to the unlocked list.

```
def draw_recipe_detail(self, screen):
```
• View Source

Draw the recipe detail overlay on top of the recipe menu.

Shows the step-by-step ingredient chain for the selected recipe, explaining which ingredients go into which machine to produce each component needed for the final drink.

Layout: - Smaller dark box centered on screen - Info bar: recipe name on the left, ESC to close on the right - Content: one line per production step (ingredient → machine → output)

def draw_shop_screen(self, screen): • View Source

Draw the full shop overlay with the tabs, paginated item list, an info bar, and pagination dot indicator.

Layout (top to bottom): - Floating "Shop" title above the box (main-menu gold style) - Menu: tabs, info bar, items, dot indicator - Pagination arrows centered vertically on the left/right edges of the box

def draw_money(self, screen, font): • View Source

Draw the current money total in the HUD.

def draw_message(self, screen): • View Source

Draw the current temporary on-screen message.

def draw_menu_screen(self, screen): • View Source

Draw the main menu with Start and Load options.

def draw_pause_menu(self, screen): • View Source

Draw the pause menu overlay.

def draw_load_menu(self, screen): • View Source

Draw the load menu with Load File options.

def draw_music_menu(self, screen, font): • View Source

Draw the music menu overlay

def draw_music_and_camera_buttons(self, screen, font): • View Source

Draw buttons for camera view and music

def change_counters_pos(self, view): • View Source

def front_view_rendering(self, player, customers, font, keys, DebugMode, game_seconds): • View Source

def middle_view_rendering(self, player, font, keys, DebugMode): • View Source

def back_view_rendering(self, player, font, keys, DebugMode): • View Source

def save_game(self, filename): • View Source

Save the current game state to a JSON file.

def load_game(self, filename): • View Source

Load the game state safely.

def reset_day(self, player): • View Source

Reset the game state for a new day within the same load.

def reset_new_game(self): • View Source

Reset the entire game state for starting a new game.

def delete_save_file(self, save): • View Source

Delete the current save file if it exists.

def end_of_day_sequence(self, screen, font): • View Source

def play_music_group(self, group_name): • View Source

Helper function for making music selections

def player_next_song(self): • View Source

Helper function for playing the next song in playlist

def main(): • View Source

items

• View Source

class Ingredient(constants.GameObject): • View Source

Represents individual items used in recipes or stored in inventory.

Attributes: name (str): The display name of the ingredient. image (pygame.Surface): The current sprite for rendering. price (float): Cost to purchase the ingredient. an_input (bool): True if the item is a raw material for a machine. stackable (bool): Determines if multiple units occupy one inventory slot.

Ingredient(name, image_keys, an_input=**False**, price_to_buy=0.0, quantity=0) • View Source

Sets up ingredient properties and pulls the initial sprite from the library.

image_keys

image

name

price

an_input

quantity

stackable

def render(self, screen): • View Source

Blits the ingredient's sprite at its current coordinates.

Inherited Members

constants.GameObject color, rect

class Cup(constants.GameObject): • View Source

An object used to hold brewed ingredients.

Attributes: contents (list): List of ingredients currently inside the cup. stackable (bool): Tracks if the cup can be stacked in inventory.

Cup(image_keys, contents=**None**) • View Source

Initializes cup with empty or pre-filled contents.

image_keys

image

name

max_capacity

def update(self): • View Source

Updates cup state and visuals based on contents.

def render(self, screen): • View Source

Blits the cup image to the screen.

Inherited Members

constants.GameObject color, rect

machines

• View Source

class Machine(constants.GameObject, pygame.sprite.Sprite): • View Source

Represents a functional appliance in the cafe, such as an Espresso Machine or Grinder.

The machine follows a state-based lifecycle: empty -> full -> running -> ready. It includes a 'mini_game_mode' for player interaction and internal timers to process ingredients into outputs.

Machine(
x,
y,
name,
machine_input,
outputs: list,

```
    num_outputs,
    runtime,
    mini_game_img_keys,
    sound_keys,
    start_button_info,
    w=150,
    h=90
)
```

• View Source

Initializes the machine with its processing rules and visual assets.

Args: x (int): X-coordinate on the counter. y (int): Y-coordinate on the counter. name (str): The display name of the machine. machine_input (Ingredient): The specific ingredient type this machine accepts. outputs (list): Possible items this machine can produce. num_outputs (int): How many items are produced per cycle. runtime (int): Processing time in seconds. mini_game_img_keys (list): Keys for empty, running, and ready sprites. start_button_info (list): [x, y, w, h] for the minigame start button. w (int): Width of the machine's footprint. h (int): Height of the machine's footprint.

state

mini_game_img_keys

sound_keys

filled_effect

brewing_effect

ready_effect

collect_effect

error_effect

counter_space_rect

rect

name

input

outputs

contents

timer_start

error_start

selected_output

interaction_zone

start_button

ingredient

ingredient_rect

placed

def get_sprite(self): • View Source

Returns the appropriate sprite from IMAGE_LIBRARY based on current state.

def is_player_nearby(self, player): • View Source

Returns True if the player's feet are within the machine's interaction zone.

def setup_minigame(self, ingredient_list): • View Source

Prepares the local ingredient reference for the minigame view.

def mini_game_mode(self, screen, debug, font): • View Source

Handles the specialized full-screen UI rendering for machine interaction.

This includes scaling the machine sprite, drawing the background, and showing timer progress.

def add(self, ingredient, player): • View Source

Attempts to load an ingredient into the machine. Sets state to 'error' if input is invalid.

def run_machine(self, num=0): • View Source

Starts the production cycle if the machine is currently full.

def begin_timer(self): • View Source

Captures start ticks and transitions machine to the running state.

def update(self): • View Source

Monitors the active timer. Transitions state to 'ready' and spawns contents once runtime expires.

def select_output(self, index=0): • View Source

Determines which output from the list will be produced.

def remove_output(self): • View Source

Returns one item from the machine's contents if the state is 'ready'.

def move_to(self, x, y): • View Source

Reposition the machine on the counter, updating all dependent rects.

def render(self, screen, debug=**False**): • View Source

Draws the machine in the standard cafe view.

Inherited Members

constants.GameObject color

others

• View Source

class Register(constants.Counter): • View Source

A specialized Counter that handles order-taking and customer queue logic.

Attributes: interaction_zone (pygame.Rect): The area where the player can trigger interaction. icon (pygame.Surface): The visual indicator for a waiting customer.

Register(x, y, iz_y, w=150, h=90) • View Source

Initializes position, dimensions, and the pygame Rect object.

customer_waiting = False

placeable

interaction_zone

icon

icon_rect

bell

order_screen

customer_image

customer_rect

def set_waiting(self): • View Source

Sets the shared customer_waiting flag to True.

def take_order(self, screen, current_cust=**None**): • View Source

Draws the register order-taking screen and UI elements.

Args: screen (pygame.Surface): The display surface. current_cust (Customer): The customer being served.

def render(self, screen): • View Source

Renders the register icon in the game world.

Inherited Members

constants.GameObject color, rect

class Sink(constants.Counter): • View Source

A specialized Counter that handles clearing the player's inventory cup.

Attributes: interaction_zone (pygame.Rect): Area where player can interact with the sink.

Sink(x, y) • View Source

Initializes position, dimensions, and the pygame Rect object.

placeable

interaction_zone

def clear_cup(self, player): • View Source

Empties the cup in the player's active inventory slot.

Args: player (Player): The player instance performing the action.

def is_player_nearby(self, player): • View Source

Checks if the player is within range of the sink.

def render(self, screen, debugmode=**False**): • View Source

Renders the sink unit visuals.

Inherited Members

constants.GameObject color, rect

player

• View Source

class Player(constants.GameObject, pygame.sprite.Sprite):

• View Source

The main controllable entity in the cafe.

Attributes: sprite (pygame.Surface): The current visual image of the player. rect (pygame.Rect): The collision and position rectangle. foot_w (int): The width of the specialized foot collision box. foot_h (int): The height of the specialized foot collision box. selected_slot (int): The index of the currently active inventory slot. inventory (list): A 2D list containing item objects for each slot. inventory_quants (list): A list tracking the quantity of items in each slot.

Player(x: int, y: int) • View Source

Initializes the player with a sprite and an empty 4-slot inventory.

Args: x (int): Starting horizontal position. y (int): Starting vertical position. image_key (str): Key to retrieve the player sprite from IMAGE_LIBRARY.

direction

is_moving

current_frame

animations

image

rect

foot_w

foot_h

selected_slot

inventory

inventory_quants

speed

max_stack_size

def get_foot_rect(self): • View Source

Returns a rectangle that only covers the feet area of the player sprite.

Returns: pygame.Rect: A rectangle located at the bottom-center of the player.

def handle_movement(self, keys, collisions): • View Source

Updates the player position if keys are pressed and handles collisions.

Args: keys (pygame.key.ScancodeWrapper): The state of all keyboard buttons. collisions (list): A list of Rect objects to check against for collisions.

def update_inv_lengths(self): • View Source

Updates the inventory_quants list with current lengths of inventory slots.

Uses dynamic slot count so new slots added by the More Pockets upgrade are tracked automatically without needing to update this method.

def add_item_to_inv(self, item, item_type): • View Source

Adds a given object to the player's inventory, handling stacking logic.

Stacking respects self.max_stack_size, which can be increased via the Deeper Pockets shop upgrade. Slot count is dynamic — self.inventory may have more than the base 4 slots if More Pockets has been purchased.

Args: item (GameObject): The object to be added. item_type (type): The class type of the item for comparison.

def pop_inv_item(self, item, type): • View Source

Removes a given object from the player's inventory and returns it.

Args: item (GameObject): The specific instance to remove. type (type): The class type of the item.

def animate(self): • View Source

Switches the self.image based on direction and movement.

def render(self, screen, debugmode): • View Source

Renders the player sprite and optional debug collision box.

Args: screen (pygame.Surface): The display surface to draw on. debugmode (bool): Whether to draw the foot collision rectangle.

Inherited Members

constants.GameObject color

recipes

• View Source

class Recipe: • View Source

The Recipe class defines a recipe a customer can order and a player can make.

This class manages the required ingredients, pricing, and the locked/unlocked status of specific drinks in the cafe.

Recipe(name: str, ingredients: list, price: float, image_key, locked=**True**) • View Source

A recipe includes a name, a list of ingredients, a price, and a sprite.

Args: name (str): The name of the recipe. ingredients (list): List of ingredient objects required. price (float): The cost of the recipe. image_key (str): The key to look up the image in IMAGE_LIBRARY. locked (bool): Whether the recipe is available to the player.

ingredients

locked

def check_match(self, cup: list): • View Source

Will take the ingredients of the current drink and try to match it to whatever recipe the customer has as their ordered recipe.

Is called when trying to deliver and returns True or False.

def get_name(self): • View Source

Returns name of recipe.

def get_ingredients(self): • View Source

Returns the ingredient list to create recipe, in order.

def get_price(self): • View Source

Returns the price of the recipe.

def get_status(self): • View Source

Returns whether the recipe is currently locked or not.

def set_status(self, state): • View Source

Sets the recipe state as either True (recipe locked) or False (recipe unlocked).

def str(self): • View Source

Returns all attributes of the recipe in a sentence.

def render(self): • View Source

Renders the recipe image. Used in recipe menu.